KDL OS Installation Guide

F9 Tech
October 26, 2026

CONTENTS

KDL OS — Installation Guide

Quick start (for people in a hurry)

1. What you are installing

2. Choose how to install

3. Before you start

3.1 Software to install on your computer

3.2 Your computer

3.3 Accounts and keys — none needed to start

3.4 What Docker installs for you (do not install these by hand)

3.5 Ports at a glance

Windows users — start here

4. Path A — Install with Docker

Step 1 — Install Git and Docker

Step 2 — Get the code

Step 3 — Create the .env file

Step 4 — Build and start everything

Step 5 — Wait until it is ready

Step 6 — Create the first admin user (“seed”)

Step 7 — Open KDL OS and log in

Step 8 — Check that it works

You now have

5. Everyday commands (Docker)

6. Path B — Developer mode

Step B1 — Prepare

Step B2 — Start only the databases and storage (terminal 1)

Step B3 — Backend (terminal 1)

Step B4 — AI services (terminal 2)

Step B5 — Frontend (terminal 3)

Step B6 — Check

Optional — run the automated tests

7. Optional setup

7.1 Add real AI keys

7.2 Finish the search setup (MeiliSearch keys)

7.3 Other features

8. Troubleshooting

9. Uninstall or start over

Appendix A — Why this guide differs from the older docs

Appendix B — Where to read more

KDL OS — Installation Guide

How to install and run KDL OS from the Git repository, step by step — including everything you must install first.

Who this is for	Anyone setting KDL OS up for the first time — developers, testers, designers. You do not need to know the project.
Repository	https://github.com/F9info/kdl-osSMARTPANTS_PRESERVED_291 — public, default branch `master`
Written against	`master` at commit `e86ab87`, 2026-10-24
Time needed	About 30–45 minutes, most of it waiting for the first build (an estimate)
Result	KDL OS running on your own computer at http://localhostSMARTPANTS_PRESERVED_317, with an admin login

> **Heads-up:** `README.md`, `docs/SETUP.md` and `setup.sh` in the repository are partly out of date (wrong ports, missing variables). Follow this guide instead. Details are in Appendix A.

Quick start (for people in a hurry)

You need Git, Docker (running) and OpenSSL. Then:

1. Get the code

```
git clone https://github.com/F9info/kdl-os.git
cd kdl-os
```

2. Create the `.env` file — copy the block from Step 3 below.

3. Build, start, and create the first admin user (the first build takes several minutes)

```
docker compose up -d --build
docker compose exec backend node prisma/seed.js
```

4. Log in at http://localhost/loginSMARTPANTS_PRESERVED_353 with `admin@kdl.com` / `kdl-dev-seed-password`. You will be asked to choose a new password.

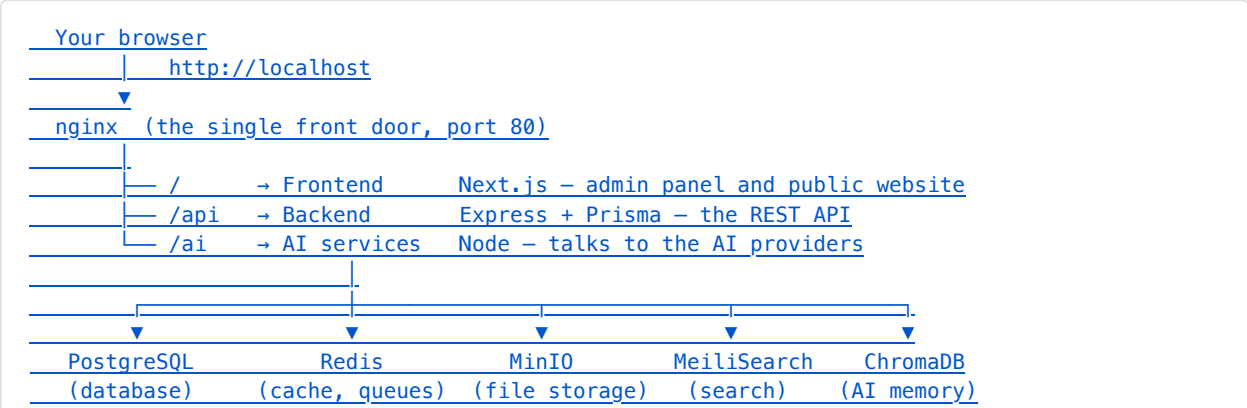
The rest of this guide explains each step and what to do when something goes wrong.

---

1. What you are installing

KDL OS (the repository is also called *KDL Starter Kit*) is a ready-made SaaS foundation: an admin panel with login and roles, media and content management, a page builder, brand and theme settings, and AI-assisted website generation.

It is not one program. It is a set of services that work together. Docker starts all of them for you:



---

2. Choose how to install

	Path A — Docker (recommended)	**Path B — Developer mode**
Best for	Running KDL OS: testers, designers, managers, and developers who just need it up	Developers who edit code and want live reload
Where things run	Everything in containers	Databases in Docker; backend, frontend and AI services run directly on your computer
You install	Git, Docker	Git, Docker, Node.js, pnpm
You open	http://localhostSMARTPANTS_PRESERVED_414SMARTPANTS_PRESERVED_415	http://localhost:3000SMARTPANTS_PRESERVED_418SMARTPANTS_PRESERVED_419

Both paths use the same `.env` file (Step 3). **Do not run both at the same time** — they use the same database ports.

If you are unsure, choose **Path A**.

---

3. Before you start

3.1 SOFTWARE TO INSTALL ON YOUR COMPUTER

Software	Path A	Path B	Version	Check it	Where to get it
Git	Yes	Yes	any recent	`git --version`	https://git-scm.comSMARTPANTS_PRE-SERVED_469SMARTPANTS_PRESERVED
Docker — Docker Desktop on Windows/Mac, Docker Engine + Compose plugin on Linux	Yes	Yes	recent, with **Compose v2** (`docker compose` , with a space). Known to work: Docker 29.6 / Compose 5.3	`docker --version` and `docker compose version`	https://www.docker.com/products/docker-desktopSMARTPANTS_PRE-SERVED_489SMARTPANTS_PRESERVED
OpenSSL	Yes	Yes	any	`openssl version`	Already on macOS and Linux. On Windows use WSL
Node.js	—	Yes	20 LTS or newer (CI and the backend image use 20; the frontend image uses 22)	`node -v`	https://nodejs.orgSMARTPANTS_PRE-SERVED_523SMARTPANTS_PRESERVED
npm	—	Yes	comes with Node	`npm -v`	—
pnpm	—	Yes (frontend only)	9 or newer (the Docker build pins 11.1.2)	`pnpm -v`	`npm install -g pnpm`
ffmpeg, qpdf, tesseract, poppler	—	Optional	any	—	macOS: `brew install ffmpeg qpdf tesseract poppler` · Ubuntu: `sudo apt install ffmpeg qpdf tesseract-ocr poppler-utils`

The last row is only needed in Path B, and only for media editing, PDF and OCR features. The Docker image already contains them.

Windows: follow the step-by-step Windows users — start here section first (WSL 2 + Ubuntu + Docker Desktop). Then run every command in the Ubuntu terminal.

How to install Docker Desktop (Windows and Mac):

1. Go to https://www.docker.com/products/docker-desktopSMARTPANTS_PRESERVED_589 and click **Download Docker Desktop**.
2. Pick your system: **Mac with Apple chip**, **Mac with Intel chip**, or **Windows**. (Mac: Apple menu → About This Mac shows which chip you have.)
3. Open the downloaded file and install it.
   - **Mac:** drag the Docker icon into **Applications**.
   - **Windows:** run the installer and keep **Use WSL 2** ticked. Then continue with the Windows section.
4. Start **Docker Desktop** from Applications (Mac) or the Start menu (Windows). Accept the terms. You can skip signing in.
5. Wait until the whale icon says **Docker Desktop is running**.
6. Check it: open a terminal and run `docker --version` and `docker compose version`. Both must print a version.

You must start Docker Desktop each time before you use KDL OS (unless you set it to start at login).

Linux: let your user run Docker without `sudo`: `sudo usermod -aG docker $USER`, then log out and back in.

3.2 YOUR COMPUTER

	Needed
Operating system	macOS (Intel or Apple Silicon), Linux, or Windows 10/11 with WSL 2
Memory given to Docker	**at least 6 GB, 8 GB recommended** — the frontend build asks for a 4 GB memory heap. Docker Desktop → Settings → Resources → Memory
Disk space	About 5 GB for images; keep **15 GB free** to be safe (build cache grows)
Internet	Needed on first run to download images and packages (a few GB)
Free ports	See the ports table. Port **80** must be free for Path A

3.3 ACCOUNTS AND KEYS — NONE NEEDED TO START

KDL OS starts and you can log in with **no external accounts**. These are optional and can be added later (see section 7):

- OpenRouter and/or Anthropic API keys — real AI answers (without them, AI features run in a built-in "local" fallback mode)
- OpenAI key — voice transcription
- Stripe or Razorpay keys — payments
- Sentry DSN — error tracking

- Google Drive / Dropbox / OneDrive app credentials — cloud import

In the Docker setup, outgoing email never leaves your computer: it is caught by MailHog (http://localhost:8025SMARTPANTS_PRESERVED_710).SMARTPANTS_PRESERVED_711

3.4 WHAT DOCKER INSTALLS FOR YOU (DO NOT INSTALL THESE BY HAND)

Service	Image / runtime	What it does
PostgreSQL 15	`postgres:15`	Main database
Redis 7	`redis:7-alpine`	Cache, login lockout counters, background-job queues. Password-protected
MinIO	`minio/minio` (latest — not pinned)	File and media storage (S3-compatible)
MeiliSearch 1.7	`getmeili/meilisearch:v1.7`	Full-text search
ChromaDB	`chromadb/chroma` (latest — not pinned)	Vector store for the AI knowledge base
MailHog 1.0.1	`mailhog/mailhog:v1.0.1`	Catches outgoing email so nothing real is sent
nginx	`nginx:alpine`	Single front door on port 80
Backend	built from `backend/` — Node 20, Express 5, Prisma 7, BullMQ	The API. Creates the database tables on start. The image also contains ffmpeg, qpdf, tesseract and poppler
Frontend	built from `frontend/` — Node 22, Next.js 15, React 18, Tailwind 3	Admin panel and public website
AI services	built from `ai-services/` — Node 20	Talks to Claude / OpenRouter, does AI search
Site preview	`node:22-alpine`	Live preview of generated websites; sits idle until a site is generated
ClamAV	`clamav/clamav`	Virus scanning — **off by default** (optional)

The programming-language packages (Express, Prisma, Next.js, …) are downloaded automatically during the Docker build. The full lists are in `backend/package.json`, `frontend/package.json` and `ai-services/package.json`.

> MinIO and ChromaDB are not pinned to a version, so a fresh install pulls whatever is newest that day.

3.5 PORTS AT A GLANCE

Everything listens on `127.0.0.1` only, so nothing is reachable from other computers on your network.

Service	Path A (Docker)	Path B (Developer mode)
Website entry (nginx)	**80** → http://localhostSMARTPANTS_PRESERVED_863SMART-PANTS_PRESERVED_864	—
Frontend	3001 (direct, bypassing nginx)	3000
Backend API	4000	4000
AI services	5001	5000
PostgreSQL	5433	5433
Redis	6380	6380
MinIO API / console	9002 / 9003	9000 / 9001
MeiliSearch	7700	7700
ChromaDB	8000	8000
MailHog web UI (SMTP 1025)	8025	—
Site preview	3300	—

Windows users — start here

Windows needs three extra things before Path A. Do them once, in this order. After that, every command in this guide is typed in the **Ubuntu** window, not in PowerShell or Command Prompt.

W1. Install Ubuntu (WSL 2). WSL is a built-in Windows feature that gives you a Linux terminal.

1. Click Start, type `PowerShell`, right-click it and choose **Run as administrator**.
2. Type this and press Enter:

```
wsl --install
```

3. Restart the computer when it asks.
4. An **Ubuntu** window opens by itself after the restart. Wait for it, then type a username and a password (you will not see the password as you type). Remember them.

W2. Install Docker Desktop.

1. Download it from https://www.docker.com/products/docker-desktopSMARTPANTS_PRESERVED_984 and install it. Keep the option **Use WSL 2** ticked.
2. Start Docker Desktop and wait until it says it is running.

3. Open Docker Desktop → **Settings** → **Resources** → **WSL integration**, switch **Ubuntu** on, and click **Apply & restart**.
4. Give it memory: create the file `C:\Users\<your name>\.wslconfig` with these lines, then run `wsl --shutdown` in PowerShell and start Docker Desktop again:

```
[wsl2]
memory=8GB
```

W3. Check Ubuntu has what it needs. Open **Ubuntu** from the Start menu and type:

```
git --version
openssl version
docker compose version
```

All three must print a version. If `git` or `openssl` is missing, install them with:

```
sudo apt update
sudo apt install -y git openssl
```

If `docker compose` is not found, repeat W2 step 3.

Then follow Path A (section 4) from Step 2. Two Windows-only reminders:

- Stay inside the Ubuntu window and clone into the Ubuntu home folder: `cd ~` first. Do not clone under `/mnt/c/` — it is very slow and can break the build.
- Open the app in your normal Windows browser at http://localhost/loginSMARTPANTS_PRE-SERVED_1020. It works from Windows even though KDL OS runs inside Ubuntu.

For developer mode (Path B), also install Node.js and pnpm **inside Ubuntu** (not the Windows versions). The simplest way:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo npm install -g pnpm
```

---

4. Path A — Install with Docker

STEP 1 — INSTALL GIT AND DOCKER

Install the software in section 3.1, then start Docker Desktop and wait until it says it is running. Check:

```
git --version
docker --version
docker compose version
openssl version
```

All four must print a version. Then give Docker enough memory (section 3.2).

STEP 2 — GET THE CODE

```
git clone https://github.com/F9info/kdl-os.git
cd kdl-os
```

The repository is public, so no login is needed. `master` is checked out automatically.

> Clone into a normal folder such as `~/projects`. Avoid folders synced by iCloud Drive, OneDrive or Dropbox — sync tools can damage the hidden `.git` folder.

All remaining commands are run **inside this `kdl-os` folder** — the one that contains `docker-compose.yml`.

STEP 3 — CREATE THE `.ENV` FILE

`.env` holds passwords and settings. It is not in Git, so you create it. Copy and paste this whole block into your terminal. It writes `.env` with fresh random passwords and secrets:

```
PG_PASS=$(openssl rand -hex 16)
REDIS_PASS=$(openssl rand -hex 16)
MINIO_PASS=$(openssl rand -hex 16)

cat > .env <<EOF
# ── Core ──────────────────────────────────────────────
NODE_ENV=development
APP_PORT=4000
FRONTEND_URL=http://localhost
BACKEND_URL=http://localhost:4000
AI_SERVICES_URL=http://localhost:5000
CORS_ORIGIN=http://localhost,http://localhost:3000,http://localhost:3001

# ── Secrets (random, generated when this file was created) ──
JWT_SECRET=$(openssl rand -hex 32)
JWT_REFRESH_SECRET=$(openssl rand -hex 32)
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
APP_ENCRYPTION_KEY=$(openssl rand -hex 32)

# ── PostgreSQL ────────────────────────────────────────
POSTGRES_USER=postgres
POSTGRES_PASSWORD=$PG_PASS
POSTGRES_DB=kdl_db
DATABASE_URL=postgresql://postgres:$PG_PASS@localhost:5433/kdl_db

# ── Redis ─────────────────────────────────────────────
REDIS_PASSWORD=$REDIS_PASS
REDIS_URL=redis://:$REDIS_PASS@localhost:6380

# ── MinIO (file storage) ──────────────────────────────
MINIO_ROOT_USER=kdl_minio
MINIO_ROOT_PASSWORD=$MINIO_PASS
MINIO_ACCESS_KEY=kdl_minio
MINIO_SECRET_KEY=$MINIO_PASS
MINIO_ENDPOINT=localhost
MINIO_PORT=9000
MINIO_USE_SSL=false
MINIO_BUCKET=kdl-media

# ── MeiliSearch + ChromaDB ────────────────────────────
MEILISEARCH_HOST=http://localhost:7700
MEILI_MASTER_KEY=$(openssl rand -hex 24)
MEILISEARCH_API_KEY=replace-after-first-boot
MEILI_SEARCH_API_KEY=replace-after-first-boot
CHROMADB_URL=http://localhost:8000

# ── AI keys: "local" = built-in fallback, no key needed to boot ──
ANTHROPIC_API_KEY=local
OPENROUTER_API_KEY=local
EOF
chmod 600 .env
```

Check it worked: `ls -la .env` should show a file of about 2 KB.

What to know about this file:

- **Works on macOS, Linux and WSL / Git Bash.** It does not work in Windows PowerShell.
- **Do not just copy `.env.example`.** It contains placeholders, and the backend refuses to start with them: the JWT secrets must be at least 32 characters and must not look like placeholders, and `APP_ENCRYPTION_KEY` must be exactly 64 hex characters.
- **Keep it private.** Never commit it or send it to anyone. Git already ignores it.
- The same file works for Docker (Path A) and developer mode (Path B). Inside the containers, Docker swaps `localhost` for the right service names (database, Redis, MinIO, …). One exception is in section 7.2.
- `MEILISEARCH_API_KEY` is a placeholder. The app starts fine, but search needs real keys — see section 7.2.
- The `*_API_KEY=local` lines mean “use the built-in AI fallback”. Real keys come later (section 7.1).

STEP 4 — BUILD AND START EVERYTHING

```
docker compose up -d --build
```

This downloads the ready-made services, builds KDL’s own backend, frontend and AI services from the source code, and starts everything in the background. The first run takes several minutes (about 10–20 on an average machine, estimated) — the Next.js build is the slow part. Later starts take seconds.

The backend creates all database tables automatically when it starts. You do not run migrations yourself.

If it stops with an error, look at Troubleshooting.

STEP 5 — WAIT UNTIL IT IS READY

```
docker compose ps
```

Look at the **STATUS** column:

Service	Should say
postgres, redis, backend, ai-services	`Up … (healthy)` — the backend can take about a minute to become healthy
frontend, nginx, minio, meilisearch, chromadb, mailhog	`Up`
site-preview	`Up` — it just waits until a website is generated; that is normal

To watch the backend start: `docker compose logs -f backend` (press `Ctrl+C` to stop watching). Wait for the line `Backend running on port 4000`.

STEP 6 — CREATE THE FIRST ADMIN USER ("SEED")

The new database is empty. "Seeding" fills it with the starter data: the admin account, roles and permissions, the built-in modules, a default project, brand and website settings, and the Theme Engine defaults.

```
docker compose exec backend node prisma/seed.js
```

You should see `Seeded admin: admin@kdl.com`. It is safe to run again — it never overwrites an existing admin.

To choose your own starting password, add `SEED_ADMIN_PASSWORD=<at least 12 characters>` to `.env` **before** this step.

STEP 7 — OPEN KDL OS AND LOG IN

1. Open http://localhost/loginSMARTPANTS_PRESERVED_1150SMARTPANTS_PRESERVED_1151.SMARTPANTS_PRESERVED_1152
2. Email: `admin@kdl.com`
3. Password: `kdl-dev-seed-password`
4. KDL OS now asks you to set a new password. This is expected: seeded passwords are temporary and nothing else works until you change it. Choose a strong one and keep it somewhere safe.

After that you land in the admin panel (`/admin`).

First-run question. Every admin sees a pop-up: *"Do you need a front-end website?"*

- Yes, create my website — KDL OS creates and publishes 5 pages (Home, About, Services, Contact, FAQ) with neutral colours and "Your Brand" text. Open them at `/p/home` , `/p/about` , and so on. Edit them in Page Builder and change colours in Theme Engine.
- No, backend only — nothing is created. You can add pages later in Page Builder.
- Ask me later — closes the pop-up for this browser session; it returns next time until someone answers.

Add content features: install the CMS module. Open Modules, find CMS, click Install (one click). It adds FAQ, Gallery, Testimonials, Team, Products and services, Our Work, Sectors, Case studies and Menus together, with sample content, and shows them in the admin menu. If you chose "Yes, front-end website", the site also gets these pages: Gallery, Testimonials, Team, Products, Our Work, Sectors, Case studies, and the FAQ page reads from the FAQ module. The header menu shows the main 7 pages; the footer lists them all. Pages you have edited are never overwritten. Disabling CMS switches all of its parts off and keeps your data.

STEP 8 — CHECK THAT IT WORKS

```
curl http://localhost:4000/health
curl http://localhost:5001/health
```

The first checks the backend (its answer contains `"status":"ok"`), the second the AI services. Each should return a short JSON message instead of an error.

Done when all of these are true:

- [ ] `docker compose ps` shows no service restarting or unhealthy
- [ ] http://localhost/loginSMARTPANTS_PRESERVED_1213 shows the login page
- [ ] You can log in and see the admin panel
- [ ] http://localhost:8025SMARTPANTS_PRESERVED_1221 (MailHog) opens

YOU NOW HAVE

What	Address
KDL OS (use this)	http://localhostSMARTPANTS_PRESERVED_1243SMARTPANTS_PRESERVED_1244
Admin login page	http://localhost/loginSMARTPANTS_PRESERVED_1251SMARTPANTS_PRESERVED_1252
Backend health check	http://localhost:4000/healthSMARTPANTS_PRESERVED_1259SMARTPANTS_PRESERVED_1260
MailHog — see test emails	http://localhost:8025SMARTPANTS_PRESERVED_1267SMARTPANTS_PRESERVED_1268
MinIO console	http://localhost:9003SMARTPANTS_PRESERVED_1275 (login: `MINIO_ROOT_USER` / `MINIO_ROOT_PASSWORD` from `.env`)
MeiliSearch	http://localhost:7700SMARTPANTS_PRESERVED_1283SMARTPANTS_PRESERVED_1284

---

5. Everyday commands (Docker)

Run these inside the `kdl-os` folder.

I want to…	Command
See what is running	`docker compose ps`
Stop everything (keeps your data)	`docker compose down`
Start again	`docker compose up -d`
Watch a service's logs	`docker compose logs -f backend` (also `frontend`, `ai-services`, `nginx`, `postgres`)
Restart one service	`docker compose restart backend`
Apply changes made in `.env`	`docker compose up -d`
Update to the latest code	`git pull` then `docker compose up -d --build`
Open a database shell	`docker compose exec postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'`
Reset a forgotten admin password	`docker compose exec backend node scripts/reset-admin-password.js --email admin@kdl.com --password 'YourNewPassword12'` (12+ characters)
Delete everything and start over	See section 9

After `git pull`, new database changes are applied automatically when the backend restarts.

---

6. Path B — Developer mode

Use this if you will change the code and want live reload. The databases still run in Docker; the three apps run on your computer, so you need three terminal windows.

STEP B1 — PREPARE

Do Steps 1–3 of Path A (install the software, clone, create `.env`). In Step 1 also install Node.js and pnpm from section 3.1. **Do not run `docker compose up -d --build`.**

STEP B2 — START ONLY THE DATABASES AND STORAGE (TERMINAL 1)

```
docker compose -f docker-compose.infra.yml up -d
docker compose -f docker-compose.infra.yml ps
```

This starts PostgreSQL, Redis, MinIO, MeiliSearch and ChromaDB (ports in 3.5). It does not include MailHog, so emails are not caught in this mode.

STEP B3 — BACKEND (TERMINAL 1)

```
cd backend
npm install
cp ../.env .env
mkdir -p logs
npx prisma migrate deploy
npm run db:seed
npm run dev
```

What each line does:

- `npm install` — downloads the packages and generates the Prisma database client
- `cp ../.env .env` — the backend reads `.env` from its own folder, so it needs a copy
- `npx prisma migrate deploy` — creates the database tables
- `npm run db:seed` — creates the admin user (see the note below)
- `npm run dev` — starts the API on http://localhost:4000SMARTPANTS_PRESERVED_1407 and keeps running

Note on the seed: here, no password is pinned. Unless you add `SEED_ADMIN_PASSWORD=<12+ characters>` to `.env` first, the seed prints a random one-time password. Copy it right away — it is shown only once. If you miss it, set a new one from the `backend/` folder:

```
node scripts/reset-admin-password.js --email admin@kdl.com --password 'YourNewPassword12'
```

STEP B4 — AI SERVICES (TERMINAL 2)

```
cd ai-services
npm install
cp .env.example .env
JWT=$(grep '^JWT_SECRET=' ../.env | cut -d= -f2-)
REDIS=$(grep '^REDIS_URL=' ../.env | cut -d= -f2-)
sed -i.bak -e "s|^JWT_SECRET=.*|JWT_SECRET=$JWT|" -e "s|^REDIS_URL=.*|REDIS_URL=$REDIS|" \
  -e 's|^ANTHROPIC_API_KEY=.*|ANTHROPIC_API_KEY=local|' -e 's|^OPENROUTER_API_KEY=.*|OPENROUTER_API_KEY=local|' .env
rm .env.bak
npm run dev
```

The `JWT=` … `rm .env.bak` lines copy what this service needs from the root `.env`: the JWT secret, the Redis address (with its password) and the two AI-key placeholders. The AI service must use the same `JWT_SECRET` as the backend so they accept each other's login tokens. The last line starts it on http://localhost:5000SMARTPANTS_PRESERVED_1422.SMARTPANTS_PRESERVED_1423

STEP B5 — FRONTEND (TERMINAL 3)

```
cd frontend
pnpm install
BACKEND_INTERNAL_URL=http://localhost:4000 pnpm dev
```

Open http://localhost:3000/loginSMARTPANTS_PRESERVED_1429SMARTPANTS_PRESERVED_1430. The frontend forwards `/api` calls to the backend; `BACKEND_INTERNAL_URL` tells it where the backend is.

(`pnpm install` also installs the project's Git hooks. That is normal.)

STEP B6 — CHECK

```
curl http://localhost:4000/health
curl http://localhost:5000/health
```

OPTIONAL — RUN THE AUTOMATED TESTS

Run each block from the repository root. Backend:

```
cd backend
JWT_SECRET="ci-jwt-secret-must-be-at-least-32-characters-long" JWT_REFRESH_SECRET="ci-refresh-secret-must-be-at-least-32-characters" npm test
```

Frontend:

```
cd frontend
pnpm test
```

AI services:

```
cd ai-services
JWT_SECRET="ci-jwt-secret-must-be-at-least-32-characters-long" npm test
```

The full list of checks used in CI is in `docs/CI_LOCAL_VERIFICATION.md`.

---

7. Optional setup

7.1 ADD REAL AI KEYS

Without keys, AI features run in a reduced "local" fallback mode. To use real models, edit `.env`:

```
ANTHROPIC_API_KEY=sk-ant-...
OPENROUTER_API_KEY=sk-or-...
```

Then apply it: `docker compose up -d backend ai-services` (Path A), or restart the apps (Path B). Keys come from https://console.anthropic.comSMARTPANTS_PRESERVED_1455 and https://open-router.aiSMARTPANTS_PRESERVED_1457.SMARTPANTS_PRESERVED_1458

7.2 FINISH THE SEARCH SETUP (MEILISEARCH KEYS)

The app uses two limited keys instead of MeiliSearch's master key. The commands below are from the project's own `.env.example`. They create the keys using the master key already in your `.env`:

First, read the master key into a variable:

```
MASTER=$(grep '^MEILI_MASTER_KEY=' .env | cut -d= -f2-)
```

Then create the key for the backend:

```
curl -s -X POST http://localhost:7700/keys \
  -H "Authorization: Bearer $MASTER" -H 'Content-Type: application/json' \
  -d '{"description":"kdl backend admin","actions":
["documents.*","indexes.*","settings.*","search"],"indexes":["*"],"expiresAt":null}'
```

And the search-only key for the AI services:

```
curl -s -X POST http://localhost:7700/keys \
  -H "Authorization: Bearer $MASTER" -H 'Content-Type: application/json' \
  -d '{"description":"kdl ai search","actions":["search"],"indexes":["*"],"expiresAt":null}'
```

Each call prints JSON. Copy the value of `"key"` from the first into `MEILISEARCH_API_KEY` and from the second into `MEILI_SEARCH_API_KEY` in `.env`. (Path B: also update `backend/.env` and `ai-services/.env`.)

Docker only: also set `MEILISEARCH_HOST=http://meilisearch:7700` in `.env`. The compose file points the backend at MeiliSearch by service name, but not the AI services, so without this line they would look for MeiliSearch inside their own container and fail. (Switch it back to `http://localhost:7700` if you later move to Path B.)

Then run `docker compose up -d backend ai-services`.

7.3 OTHER FEATURES

Feature	How to switch it on
Virus scanning of uploads	Add `CLAMAV_HOST=clamav` to `.env`, then `docker compose --profile scan up -d`
Cloud storage instead of MinIO (S3, Cloudflare R2, DigitalOcean Spaces)	Set `STORAGE_DRIVER` and its keys — see the comments in `.env.example`
Payments, error tracking, voice transcription	Add the Stripe / Razorpay / Sentry / OpenAI values from `.env.example`
Import from Google Drive / Dropbox / OneDrive	Register an OAuth app with the provider and add its client id and secret — see `.env.example`

All variables are described in `docs/ENV_REFERENCE.md`.

---

8. Troubleshooting

Start with `docker compose ps`, then read the log of the service that is not healthy: `docker compose logs --tail=100 <service>`.

What you see	Why	What to do
`required variable … is missing a value` or `… is required in .env`	`.env` is missing or incomplete, or you are not in the `kdl-os` folder	Run `ls -la .env` in the folder that holds `docker-compose.yml`. If missing, redo Step 3
`port is already allocated` / `address already in use` (80, 3001, 4000, 5433, 6380 …)	Another program uses that port — for example a local PostgreSQL on 5433, or a web server on 80	Stop that program. Or move KDL's port — see "Moving a port" below the table
Backend keeps restarting or shows `unhealthy`	Almost always a bad value in `.env`; the log names it	`docker compose logs --tail=50 backend`. Typical messages: `JWT_SECRET must be at least 32 bytes`, `appears to be a placeholder`, `APP_ENCRYPTION_KEY must be a 32-byte hex string`, `CORS_ORIGIN is required`, `missing required env vars`. Fix `.env`, then `docker compose up -d backend`
Frontend build ends with `Killed`, `exit code 137` or `JavaScript heap out of memory`	Docker has too little memory	Docker Desktop → Settings → Resources → Memory: 8 GB. Then `docker compose build frontend` and `docker compose up -d`
Browser shows `502 Bad Gateway` right after start	nginx is up but the backend or frontend is still starting	Wait 1–2 minutes and refresh. Check `docker compose ps`
Login says invalid credentials	The seed was not run, the password was already changed, or you used an old password from older docs (`Admin@123` no longer works)	Run Step 6. A fresh install uses `kdl-dev-seed-password`. Forgot it? Use the reset command in section 5
After login, everything fails with `PASSWORD_CHANGE_REQUIRED` (403)	First login must set a new password	Use the change-password screen the app shows
"Account locked"	Too many wrong passwords: 5 → 1 minute, 10 → 5 minutes, 20 → 30 minutes	Wait, or run the reset command (it clears the lock)
`429 Too Many Requests`	The API rate limiter	`docker compose restart backend`

What you see	Why	What to do
Search or AI features misbehave	Placeholder search keys, or no AI keys	Section 7
Images or uploads do not load (connection refused on port 9002)	MinIO is not running, or its port changed after the frontend was built (the image address is baked in at build time)	`docker compose ps minio`. If you changed ports: `docker compose build frontend`
I changed `.env` but nothing changed	`docker compose restart` does not reread `.env`	Run `docker compose up -d`. Settings that start with `NEXT_PUBLIC_` need `docker compose build frontend`
Warning about platform `linux/amd64` for MailHog (Apple Silicon)	MailHog only ships an Intel image	Harmless — it runs emulated
Strange Git errors, or files named like `something 2` inside `.git`	The repository is in an iCloud / OneDrive / Dropbox folder	Clone again into a normal folder (Step 2)
I want a clean slate	—	Section 9

Moving a port. Create a file named `docker-compose.override.yml` next to `docker-compose.yml`. Docker reads it automatically. Example — serve KDL OS on port 8080 instead of 80:

```
services:
  nginx:
    ports: !override
      - "127.0.0.1:8080:80"
```

Run `docker compose up -d`, then use http://localhost:8080SMARTPANTS_PRESERVED_1668. The same pattern works for any service in section 3.5; the format is `"127.0.0.1:<new host port>:<container port>"`. `!override` needs a reasonably recent Compose. Do not commit this file — it is specific to your computer.

---

9. Uninstall or start over

Command	What it does
`docker compose down`	Stops everything. Your data is kept
`docker compose down -v`	⚠ Stops everything and deletes all data: database, uploaded files, search index
`docker compose down -v --rmi local`	⚠ Same, and also deletes the images that were built on your computer

To start again from scratch, run `docker compose down -v`, then repeat Steps 4–6 (keep the same `.env`; if you delete `.env` too, recreate it first).

To remove KDL OS completely, run the third command, then delete the `kdl-os` folder.

> `down -v` cannot be undone. It deletes the database with everything you created.

Appendix A — Why this guide differs from the older docs

If you read `README.md`, `docs/SETUP.md` or ran `setup.sh`, you may have seen different instructions. These are the differences, with the reason:

Older doc says	Reality (this guide)
`README.md`: `cp .env.example .env`, fill in 4 values	Not enough. The compose file also needs `POSTGRES_*`, `REDIS_PASSWORD`, `MINIO_ROOT_*` and `MEILI_MASTER_KEY`, and the backend needs a valid `APP_ENCRYPTION_KEY` and real-looking JWT secrets. Step 3 creates all of them
`README.md` / `docs/SETUP.md`: Docker ports are 3000, 5000, 9000, 9001	In the Docker stack the host ports are 3001 (frontend), 5001 (AI services), 9002 / 9003 (MinIO). 3000, 5000, 9000 and 9001 apply to developer mode
`README.md`: after `docker compose up`, run `npx prisma migrate dev`	In Docker the backend applies migrations itself at start. Only the seed is needed (Step 6)
`README.md`: default logins `postgres/postgres`, `minioadmin/minioadmin`	The Docker stack uses the passwords in your `.env`, not these
`docs/SETUP.md`: `npm install` in `frontend/`	The frontend uses pnpm
`setup.sh`	Do not run it. It is a one-time bootstrap script for the project's original AI-agent setup. It writes an `.env` that does not match the compose file, and it overwrites the tracked files `.env.example` and `docker-compose.infra.yml` with older versions
Admin password `Admin@123`	Removed. The seeded password is `kdl-dev-seed-password` in Docker, or a random one-time password in developer mode — see `docs/ADMIN_ACCESS.md`

Appendix B — Where to read more

Topic	File
How the system is built, folder map, database	`docs/ARCHITECTURE.md`
Every `.env` variable	`docs/ENV_REFERENCE.md`
Admin password, lockout, recovery	`docs/ADMIN_ACCESS.md`
API endpoints	`docs/API_REFERENCE.md`
Writing a new module	`docs/MODULE_GUIDE.md`
What is built and what is pending	`STATUS.md`
Running the CI checks on your machine	`docs/CI_LOCAL_VERIFICATION.md`